

A Mini Project Report
on

**PROJECT BASED LEARNING: PREDICTING BITCOIN
PRICES USING DEEP LEARNING**

Submitted to

**UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY**

In partial fulfillment of the requirement for the award of the degree of

**BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE and ENGINEERING**

BY

R Sivaram - [4511-19-733-008]

M Shiva - [4511-19-733-041]

B Srivani - [4511-19-733-004]

Under the Esteemed Guidance of

Mrs. M. JYOTHI RANI

Assistant Professor (C)

Dept. of CSE, UCE&T, MGU.



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY**

PANAGAL, NALGONDA – 508001

JUNE - 2022

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY COLLEGE OF ENGINEERING & TECHNOLOGY
MAHATMA GANDHI UNIVERSITY

PANAGAL, NALGONDA – 508001



CERTIFICATE

This is to certify that this project entitled “**Project Based Learning: Predicting Bitcoin Prices using Deep Learning**” is a bonafide work carried out by R Sivaram [4511-19-733008], M Shiva [4511-19-733-041], B Srivani [4511-19-733-004] in partial fulfillment of the requirements for the award of mini project in Bachelor of Technology in Computer science and Engineering from Mahatma Gandhi University, Nalgonda during the year 2021-2022. The project report satisfies the academic requirements in respect of the project work prescribed for the said Degree.

Project Guide

Mrs. M. JYOTHI RANI

Assistant Professor (C)

Dept. of CSE

UCE&T, MGU

Nalgonda

Head of the Department

Mrs. CH. SWARNALATHA

Assistant Professor (C)

Dept. of CSE

UCE&T, MGU

Nalgonda

ACKNOWLEDGEMENT

We would like to express our deep gratitude to our project guide **Mrs. M. JYOTHI RANI**, Assistant Professor (C), Department of Computer Science and Engineering, UCET, MGU, for her guidance with unsurpassed knowledge and immense encouragement.

We are grateful to **Mrs. CH. SWARNALATHA**, Head of the Department, Computer Science and Engineering, for the valuable guidance and for permitting us to carry out this project.

We would like to thank Principal **Dr. R. REKHA** for her expert guidance and encouragement at various levels of our project.

We show our gratitude to our honorable Registrar **Prof. KRISHNA RAO THUMMA** for having provided all the facilities and support.

We avail this opportunity to express our deep sense of gratitude to our honorable Vice Chancellor **Prof. CH. GOPAL REDDY**, congenial atmosphere to complete this project successfully.

We thank all **teaching faculty** of Department of CSE, whose suggestions during reviews helped us in accomplishment of our project.

We express our thanks to all those who contributed directly or indirectly for the successful completion of our project work.

With gratitude,

Our Team,

R Sivaram [4511-19-733-008]

M Shiva [4511-19-733-041]

B Srivani [4511-19-733-004]

DECLARATION

We, hereby declare that the Mini Project entitled, “**Project Based Learning: Predicting Bitcoin Prices using Deep Learning**” has been carried out by us under the guidance of **Mrs. M. Jyothi Rani, Asst. Professor**, UCE&T, MGU, Nalgonda, in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in **Computer Science and Engineering**, of the University College of Engineering & Technology, MGU during the academic year 2021-22. The work done in this dissertation report is original and it has not been submitted for any other degree in any university.

(Student Signature)

R Sivaram [4511-19-733-008]

M Shiva [4511-19-733-041]

B Srivani [4511-19-733-004]

ABSTRACT

Project based learning is the methodology in which projects drive knowledge and is used in dedicated subjects without negotiating the coverage of the required technical material. In this project we discuss the scheme and delivery of project-based learning in computer science engineering as major project which adopts undergraduate creativities and emphasizes on realworld, open-ended projects. This project fosters a wide range of abilities, not only those related to content knowledge or technical skills, but also practical skills. The goal for this innovative undergrad project is to show how a trained machine model can predict the price of a cryptocurrency if we give the right amount of data and computational power. It displays a graph with the predicted values. The most popular technology is the kind of technological solution that could help mankind predict future events. With vast amount of data being generated and recorded on a daily basis, we have finally come close to an era where predictions can be accurate and be generated based on concrete factual data. Furthermore, with the rise of the crypto digital era more heads have turned towards the digital market for investments. This gives us the opportunity to create a model capable of predicting crypto currencies primarily Bitcoin. This can be accomplished by using a series of deep learning techniques and methodologies.

CONTENTS

ACKNOWLEDGEMENT	i
DECLARATION	ii
ABSTRACT	iii
CHAPTER 1: PREAMBLE	1
1.1 Motivation for work	
1.2 Introduction	
1.3 Existing method	
1.4 Proposed Method	
1.5 Problem statement	
1.6 Objective of the Project	
CHAPTER 2: LITERATURE SURVEY	4
2.1 Introduction	
2.2 Existing System	
CHAPTER 3: THEORITICAL BACKGROUND	6
3.1 Prediction Model	
3.1.1 Recurrent Neural Networks & Long Short-Term Memory	
3.2 System Architecture	
CHAPTER 4: SYSTEM REQUIREMENT SPECIFICATION	11
4.1 System Requirements	
4.1.1 Hardware Requirements	
4.1.2 Software Requirements	
4.2 Functional Requirements	
4.3 Non-Functional requirements	
CHAPTER 5: SYSTEM ANALYSIS	14
5.1 Feasibility Study	
5.1.1 Economical Feasibility	
5.1.2 Technical Feasibility	

5.1.3 Social Feasibility	
5.2 Analysis	
5.2.1 Performance Analysis	
5.2.2 Technical Analysis	
5.2.3 Economical Analysis	
CHAPTER 6: DESIGN	17
6.1 Structure Chart	
6.2 UML Diagrams	
6.2.1 Use case diagram	
6.2.2 Class diagram	
6.2.3 Sequence diagram	
6.2.4 Activity diagram	
6.2.5 Collaboration diagram	
CHAPTER 7: IMPLEMENTATION	23
7.1 Sample Code	
CHAPTER 8: TESTING	24
8.1 Testing Methodologies	
8.1.1 White Box Testing	
8.1.2 Black Box Testing	
8.2 Unit Testing	
8.3 System Testing	
8.4 Quality Assurance	
CHAPTER 9: RESULT & PERFORMANCE ANALYSIS	29
9.1 Output snapshots	
9.2 Result Analysis	
CHAPTER 10: CONCLUSION & FUTURE WORK	34
REFERENCES	35

LIST OF FIGURES

Figure 1. Recurrent neural network (RNN)model Architecture	6
Figure 2. Long short-term memory (LSTM) model Architecture	7
Figure 3. Overall Architecture of LSTM Model	10
Figure 4. Structure chart diagram for Model Training and Prediction	17
Figure 5. Use Case Diagram for LSTM Model in System	19
Figure 6. Class Diagram for LSTM Model	19
Figure 7. Execution of LSTM Model	20
Figure 8. Activity Diagram for LSTM Model	21
Figure 9. Collaboration between modules	22
Figure 10. White box Testing Process	24
Figure 11. Black Box Testing Process	26
Figure 12. RNN's LSTM Model Initialization Implementation	29
Figure 13. LSTM Model Validation Implementation	30
Figure 14. Model Prediction Using 2 Layers	31
Figure 15. Model Prediction Using 3 Layers	32
Figure 16. Model Prediction Using 4 Layers	33

CHAPTER 1: PREAMBLE

1.1 MOTIVATION FOR WORK

In a multidisciplinary education context, project-based learning appears one of the most interesting instructional strategies which tries to engage students in authentic real- world tasks to enhance learning. In project-based learning students typically engage individually or in groups with an instructor or coach or mentor. Each of the project designs and implements an approach to understand practical professional environment in the field of computer science engineering. In this project, we have applied project-based learning to develop a bit coin pricing algorithm; in addition to technical knowledge, we also learned to manage resources and time execution and work in team.

1.2 INTRODUCTION

Bitcoin is a crypto currency used worldwide for digital payment or simply for investment purposes. Bitcoin is decentralized i.e.; it is not owned by anyone. Transactions made by bitcoins are easy as they are not tied to any country. Investment can be done through various marketplaces known as “bitcoin exchanges.” These allow people to sell/buy bitcoins using different currencies. The largest bitcoin exchange is Mt Gox. Bitcoins are stored in a digital wallet which is basically like a virtual bank account. The record of all the transactions, the timestamp data is stored in a place called Blockchain. Each block contains a pointer to a previous block of data. The data on blockchain is encrypted. During transactions the users name is not revealed, but only their wallet ID is made public.

Predicting the future is no easy task. Many have tried and many have failed. But many of us would want to know what will happen next and would go to great lengths to figure that out. Imagine the possibilities of knowing what will happen in the future! Many people may regret not buying Bitcoin back then but how were they supposed to know in the first place? This is the dilemma we now face in regards to Cryptocurrency.

We do not want to miss out on the next jump in price but we do not know when that will or will not happen. So how can we potentially solve this dilemma? Maybe machine learning can tell us the answer [1].

Deep learning models can likely give us the insight we need to learn about the future of

Cryptocurrency. It will not tell us the future but it might tell us the general trend and direction to expect the prices to move. Let's try and use these machine learning models to our advantage and predict the future of Bitcoin by coding them out in Python!

1.3 EXISTING SYSTEM

Several studies have examined the valuation of bitcoin and other cryptocurrencies. For instance, authors in [1] develop a model of bitcoin pricing and provide mixed evidence about the ability of the model to explain bitcoin prices. Authors in [2] consider the valuation of bitcoin and decentralized network assets using an equilibrium model. Several other studies analyze the implications of Blockchain and related technologies for other areas in finance. Raskin and Yermak considered the implications for central banking. Work done in [2] focusses on corporate governance. Authors in [3] investigate bitcoin mining costs. Authors in [4] provides an in-depth discussion of the mechanics of cryptocurrencies.

1.4 PROPOSED SYSTEM

Statistical methods including Logistic Regression and Linear Discriminant Analysis for Bitcoin daily price prediction with high-dimensional features achieve an accuracy of 66%, outperforming more complicated machine learning algorithms. Compared with benchmark results for daily price prediction, we achieve a better performance, with the highest accuracies of the statistical methods and machine learning algorithms of 66% and 65.3%, respectively. Deep learning models such as Long Short-term Memory for Bitcoin 5- minute interval price prediction is superior to statistical methods, with better accuracy reaching. Our investigation of Bitcoin price prediction can be considered a pilot study of the importance of the sample dimension in machine learning techniques.

1.5 PROBLEM STATEMENT

Time Series forecasting & prediction plays an important role in data analysis Time series analysis is a specialized branch of statistics used extensively in fields such as Econometrics & Operation Research. Time Series is being widely used in analytics & data science. Bitcoin prices are volatile in nature and price depends on various factors. The main aim of this project is to predict Bitcoin prices using RNN's Long short-term memory (LSTM).

1.6 OBJECTIVE OF THE PROJECT

The objectives of the “Bitcoin Price Prediction” can be stated as follows:

1. Make use of machine learning algorithms to increase the accuracy of Bitcoin price prediction.
2. Allow the investors to invest wisely in bitcoin trading as the prices of bitcoin have gone up to an exaggerating amount in the last ten years.

CHAPTER 2: LITERATURE SURVEY

2.1 INTRODUCTION

"What other people think" has always been an important piece of information for most of us during the decision-making process. The Internet and the Web have now (among other things) made it possible to find out about the opinions and experiences of those in the vast pool of people that are neither our personal acquaintances nor well-known professional critics — that is, people we have never heard of. And conversely, more and more people are making their opinions available to strangers via the Internet. The interest that individual users show in online opinions about products and services, and the potential influence such opinions wield, is something that is driving force for this area of interest. And there are many challenges involved in this process which need to be walked all over in order to attain proper outcomes out of them. In this survey we analyzed basic methodology that usually happens in this process and measures that are to be taken to overcome the challenges being faced.

2.2 EXISTING SYSTEM

2.1.1 Forecasting stock market movement direction with support vector machine

The research work done by W. Hunag, Y. Nakamori, S. Y. Wang in Computers and Operations Research field. Financial time series forecasting could be beneficial for individual as well as institutional investors. But the high noise and complexity residing in the financial data make this job extremely challenging. Over the years, many researchers have used support vector regression (SVR) quite successfully to conquer this challenge. In this paper, an SVR based forecasting model is proposed which first uses the principal component analysis (PCA) to extract the low-dimensional and efficient feature information, and then uses the independent component analysis (ICA) to preprocess the extracted features to nullify the influence of noise in the features. Experiments were carried out based on 16 years' historical data of three prominent stocks from three different sectors listed in Dhaka Stock Exchange (DSE), Bangladesh.

The predictions were made for 1 to 4 days in advance targeting the short-term prediction. For comparison, the integration of PCA with SVR (PCA-SVR), ICA with SVR

(ICA-SVR) and single SVR approaches were applied to evaluate the prediction accuracy of the proposed approach. Experimental results show that the proposed model (PCA-ICA-SVR) outperforms the PCA-SVR, ICA-SVR and single SVR methods.

The Endeavor of financial time series forecasting has gained extreme attention from both the individual and institutional investors because the accurate forecasting can influence the decision behind investment. This field is characterized by data intensity, noise, non-stationary, unstructured nature, high degree of uncertainty, and hidden relationships. Capital market trend depends on many factors including political events, general economic conditions, news related to the stocks and traders' expectations. Moreover, according to academic investigations, movements in market prices are not random. Rather, they behave in a highly non-linear, dynamic manner. Therefore, predicting stock market price is a quite challenging task.

Technical analysis is a popular approach to study the capital market patterns and movement. The results of technical analysis may be a short or long-term forecast based on recurring patterns; however, this approach assumes that stock prices move in trends, and that the information which affects prices enters the market over a finite period of time, not instantaneously. Technical indicators used in this analysis are calculated from the historical trading data. Researchers use various machine learning and artificial intelligent approaches to analyse these technical indicators to predict future trends or prices. The traditional statistical models include Box Jenkins ARIMA. different problems like over-fitting or under-fitting, initializing large number of control parameters, finding the optimum solutions etc. Continuous research has introduced plentiful approaches including genetic algorithm, rough set (RS) theory, fuzzy logic and others. Most of these approaches suffer from To resolve most of these shortcomings, support vector regression has been widely used in various nonlinear regression tasks. Because, SVR uses the structural risk minimization principal for function estimation while the traditional methods implement empirical risk minimization principal.

CHAPTER 3: THEORITICAL BACKGROUND

Theoretical background highlighting some topics related to the project work is given below. The description contains several topics which are worth to discuss and also highlight some of their limitations that encourage going on finding solutions as well as highlights some of their advantages for which reason these topics and their features are used in this project.

3.1 PREDICTION MODEL

3.1.1 “RECURRENT NEURAL NETWORKS AND LONG SHORT – TERM MEMORY”

For time series data, it is well known that a recurrent neural network (RNN) model and a long short-term memory (LSTM) model is effective. Although a DNN model can be used to predict time series data, it does not exploit the temporal relationship between the data points in the input sequence. To exploit the temporal relationship, both RNN and LSTM models use an internal state (memory).

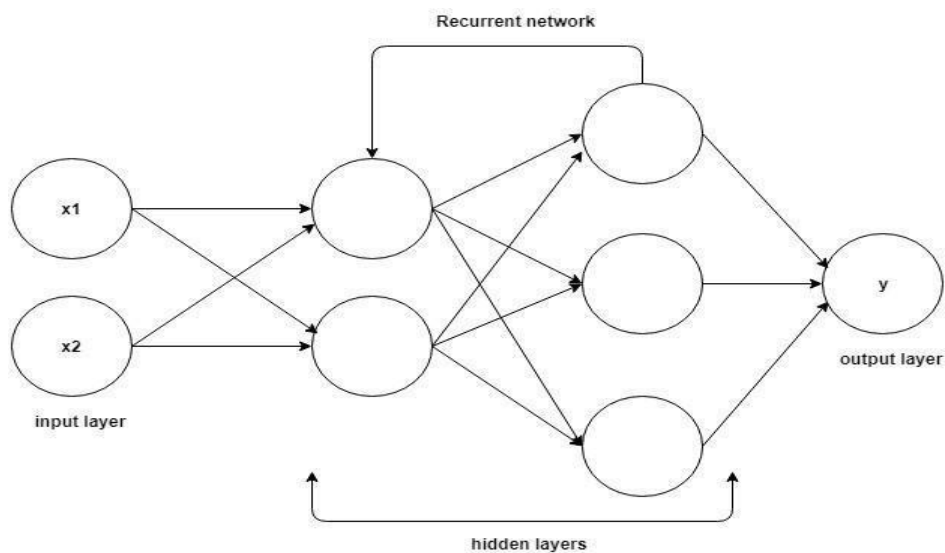


Fig. 1. Recurrent neural network (RNN)model Architecture

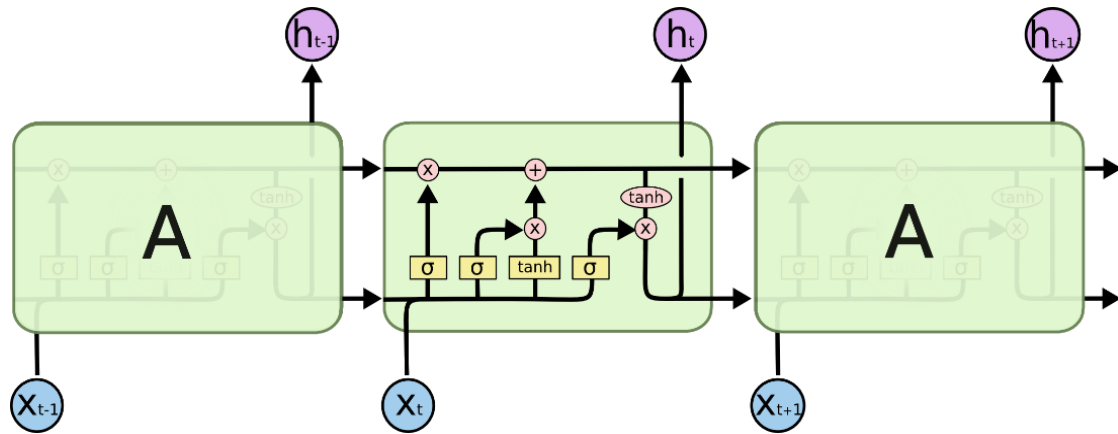


Fig. 2. Long short-term memory (LSTM) model Architecture

Long short-term memory network:

Long short-term memory network (LSTM) is a particular form of recurrent neural network (RNN).

Working of LSTM:

□

LSTM is a special network structure with three “gate” structures. Three gates are placed in an LSTM unit, called input gate, forgetting gate and output gate. While information enters the LSTM’s network, it can be selected by rules. Only the information conforms to the algorithm will be left, and the information that does not conform will be forgotten through the forgetting gate.

The experimental data in this paper are the actual historical data downloaded from the Internet. Three data sets were used in the experiments. It is needed to find an optimization algorithm that requires less resources and has faster convergence speed.

- LSTM is used instead of RNN to avoid exploding and vanishing gradients.
- The historical Bitcoin price data table contains the information of opening price, the highest price, lowest price, closing price, transaction date, volume and so on.

Forget Gate:

A forget gate is responsible for removing information from the cell state.

- The information that is no longer required for the LSTM to understand things or the

information that is of less importance is removed via multiplication of a filter.

- This is required for optimizing the performance of the LSTM network.

This gate takes in two inputs; $h(t-1)$ and $x(t)$. $h(t-1)$ is the hidden state from the previous cell or the output of the previous cell and $x(t)$ is the input at that particular time step.

Input Gate:

1. Regulating what values need to be added to the cell state by involving a sigmoid function. This is basically very similar to the forget gate and acts as a filter for all the information from $h(i-1)$ and $x(t)$.
2. Creating a vector containing all possible values that can be added (as perceived from $h(t-1)$ and $x(t)$) to the cell state. This is done using the tanh function, which outputs values from -1 to +1.
3. Multiplying the value of the regulatory filter (the sigmoid gate) to the created vector (the tanh function) and then adding this useful information to the cell state via addition operation.

Output Gate:

The functioning of an output gate can again be broken down to three steps:

- Creating a vector after applying tanh function to the cell state, thereby scaling the values to the range -1 to +1.
- Making a filter using the values of $h(t-1)$ and $x(t)$, such that it can regulate the values that need to be output from the vector created above. This filter again employs a sigmoid function.

Multiplying the value of this regulatory filter to the vector created in step 1, and sending it out as an output and also to the hidden state of the next cell.

- # LSTM Model
- Inputs: Dataset
- Outputs: Prediction Results as Graph
- # Split dataset into 80% training and 20% testing data
- size = length(dataset) * 0.8
- train = dataset [0 to size]
- test = dataset [size to length(dataset)]
- # Procedure to fit the LSTM model
- X = train
- y = test
- model = Sequential ()
- model.add(LSTM(units =50, activation ='relu', return_sequence=True))
- model.compile(optimizer='adam', loss='mse')
- model.fit(X, y, epochs=epochs, validation_split=0.2)
- return model
- #Procedure to make predictions
- Procedure getPredictionsFromModel (model, X)
- predictions = model.predict(X)
- return predictions
- epochs = 100
- neurons = 50
- predictions = empty
- # Fit the LSTM model
- model = LSTM
- Algorithm (train, epoch, neurons)
- # Make predictions
- pred = model.predict(train)

3.2 SYSTEM ARCHITECTURE

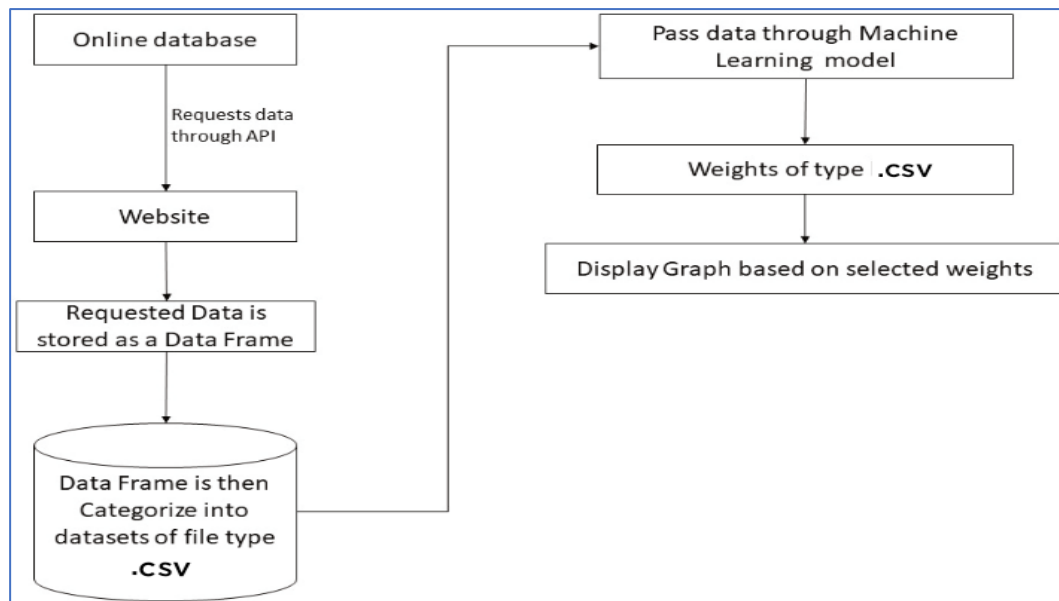


Fig. 3. Overall Architecture

CHAPTER 4: SYSTEM REQUIREMENT SPECIFICATION

A System Requirement Specification (SRS) is basically an organization's understanding of a customer or potential client's system requirements and dependencies at a particular point prior to any actual design or development work. The information gathered during the analysis is translated into a document that defines a set of requirements. It gives a brief description of the services that the system should provide and also the constraints under which the system should operate. Generally, SRS is a document that completely describes what the proposed software should do without describing how the software will do it. It's a two-way insurance policy that assures that both the client and the organization understand the other's requirements from that perspective at a given point in time.

SRS document itself states in precise and explicit language those functions and capabilities a software system (i.e., a software application, an ecommerce website and so on) must provide, as well as states any required constraints by which the system must abide. SRS also functions as a blueprint for completing a project with as little cost growth as possible. SRS is often referred to as the "parent" document because all subsequent project management documents, such as design specifications, statements of work, software architecture specifications, testing and validation plans, and documentation plans, are related to it.

Requirement is a condition or capability to which the system must conform. Requirement Management is a systematic approach towards eliciting, organizing and documenting the requirements of the system clearly along with the applicable attributes. The elusive difficulties of requirements are not always obvious and can come from any number of sources.

4.1 SYSTEM REQUIREMENTS

4.1.1 Hardware Requirements

- Processor – Intel I3 or Higher
- Speed – 1.1 GHZ
- RAM – Minimum 4GB
- Hard Disk – Minimum 50GB
- Key Board – Standard Windows Keyboard

Mouse – Two or Three Button Mouse

4.1.2 Software Requirements

- Python 3.9 or latest version, is used for data pre-processing, model training and prediction.
- Python IDE: Jupyter Notebook
- Operating System: windows 7 and above or Linux based OS or MAC OS

4.2 FUNCTIONAL REQUIREMENTS

Functional Requirement defines a function of a software system and how the system must behave when presented with specific inputs or conditions. These may include calculations, data manipulation and processing and other specific functionality.

Following are the functional requirements on the system:

1. The entire control model set must be translated to C output Code.
2. Inputs must be models designed using CLAW design components along with standard design components, Multiple design models must be processed and the result must be combined to obtain a single output file.

4.3 NON-FUNCTIONAL REQUIREMENT

Nonfunctional requirements are the requirements which are not directly concerned with the specific function delivered by the system. They specify the criteria that can be used to judge the operation of a system rather than specific behaviors. They may relate to emergent system properties such as reliability, response time and store occupancy. Nonfunctional requirements arise through the user needs, because of budget constraints, organizational policies, the need for interoperability with other software and hardware systems or because of external factors such as: -

- Product Requirements
- Organizational Requirements

4.3.1 Product Requirements

Platform Independency: Standalone executables for embedded systems can be created so

the algorithm developed using available products could be downloaded on the actual hardware and executed without any dependency to the development and modeling platform.

Correctness: It followed a well-defined set of procedures and rules to compute and also rigorous testing is performed to confirm the correctness of the data.

Ease of Use: Model Coder provides an interface which allows the user to interact in an easy manner.

Modularity: The complete product is broken up into many modules and well-defined interfaces are developed to explore the benefit of flexibility of the product.

Robustness: This software is being developed in such a way that the overall performance is optimized and the user can expect the results within a limited time with utmost relevance and correctness. Nonfunctional requirements are also called the qualities of a system.

These qualities can be divided into execution quality & evolution quality. Execution qualities are security & usability of the system which are observed during run time, whereas evolution quality involves testability, maintainability, extensibility or scalability.

4.3.2 Organizational Requirements

Process Standards: The standards defined by DRDO are used to develop the application which is the standard used by the developers inside the defense organization.

Design Methods: Design is one of the important stages in the software engineering process. This stage is the first step in moving from problem to the solution domain. In other words, starting with what is needed design takes us to work how to satisfy the needs.

CHAPTER 5: SYSTEM ANALYSIS

Analysis is the process of finding the best solution to the problem. System analysis is the process by which we learn about the existing problems, define objects and requirements and evaluate the solutions. It is the way of thinking about the organization and the problem it involves, a set of technologies that helps in solving these problems. Feasibility study plays an important role in system analysis which gives the target for design and development.

5.1 FEASIBILITY STUDY

All systems are feasible when provided with unlimited resources and infinite time. But unfortunately, this condition does not prevail in the practical world. So it is both necessary and prudent to evaluate the feasibility of the system at the earliest possible time. Months or years of effort, thousands of rupees and untold professional embarrassment can be averted if an ill-conceived system is recognized early in the definition phase. Feasibility & risk analysis are related in many ways. If project risk is great, the feasibility of producing quality software is reduced. In this case three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

5.1.1 Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and

development of the system is limited. The expenditures must be justified.

Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available.

5.1.2 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

5.1.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

5.2 ANALYSIS

5.2.1 Performance Analysis

For the complete functionality of the project work, the project is run with the help of a healthy networking environment. Performance analysis is done to find out whether the proposed system. It is essential that the process of performance analysis and definition must be conducted in parallel.

5.2.2 Technical Analysis

System is only beneficial only if it can be turned into information systems that will meet the organization's technical requirement. Simply stated this test of feasibility asks whether the system will work or not when developed & installed, whether there are any major barriers to implementation. Regarding all these issues in technical analysis there are

several points to focus on: -

Changes to bring in the system: All changes should be in a positive direction, there will be increased level of efficiency and better customer service.

Required skills: Platforms & tools used in this project are widely used. So the skilled manpower is readily available in the industry.

Acceptability: The structure of the system is kept feasible enough so that there should not be any problem from the user's point of view.

5.2.3 Economical Analysis

Economic analysis is performed to evaluate the development cost weighed against the ultimate income or benefits derived from the developed system. For running this system, we need not have any routers which are highly economical. So, the system is economically feasible enough.

CHAPTER 6: DESIGN

6.1 STRUCTURE CHART

A structure chart (SC) in software engineering and organizational theory is a chart which shows the breakdown of a system to its lowest manageable levels. They are used in structured programming to arrange program modules into a tree. Each activity is represented by a box, which contains the activity's name.

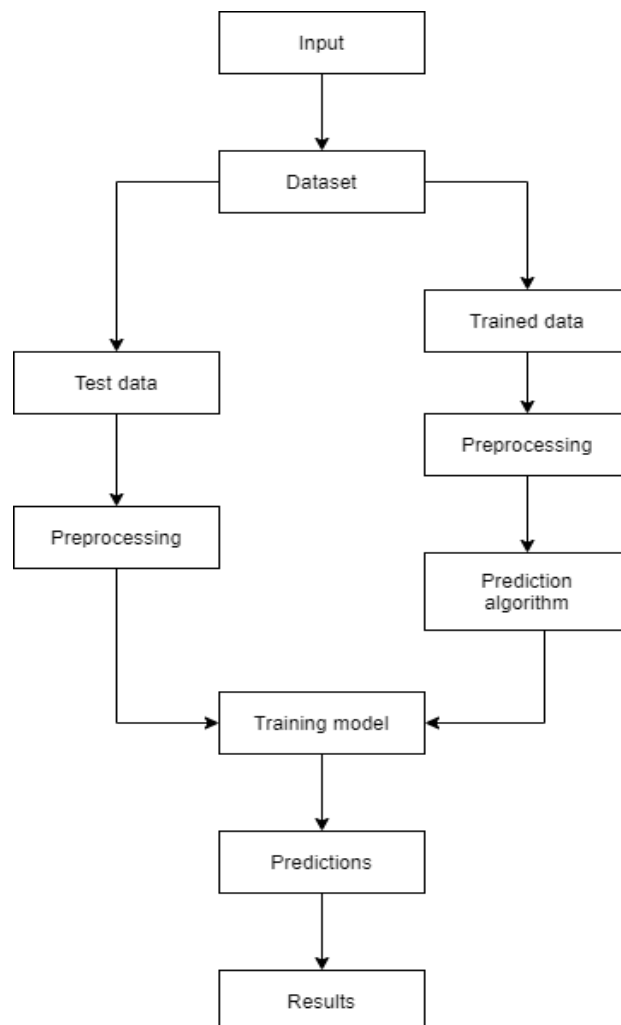


Fig. 4. Training and Prediction

6.2 UML DIAGRAMS

An UML diagram is a partial graphical representation (view) of a model of a system under design, implementation, or already in existence. UML diagram contains graphical elements (symbols) - UML nodes connected with edges (also known as paths or flows) -

that represent elements in the UML model of the designed system. The UML model of the system might also contain other documentation such as use cases written as templated texts.

The kind of the diagram is defined by the primary graphical symbols shown on the diagram. For example, a diagram where the primary symbols in the contents area are classes is class diagram. A diagram which shows use cases and actors is use case diagram. A sequence diagram shows sequence of message exchanges between lifelines.

UML specification does not preclude mixing of different kinds of diagrams, e.g. to combine structural and behavioral elements to show a state machine nested inside a use case. Consequently, the boundaries between the various kinds of diagrams are not strictly enforced. At the same time, some UML Tools do restrict set of available graphical elements which could be used when working on specific type of diagram.

UML specification defines two major kinds of UML diagram: structure diagrams and behavior diagrams.

Structure diagrams show the static structure of the system and its parts on different abstraction and implementation levels and how they are related to each other. The elements in a structure diagram represent the meaningful concepts of a system, and may include abstract, real world and implementation concepts.

Behavior diagrams show the dynamic behavior of the objects in a system, which can be described as a series of changes to the system over time.

6.2.1 Use Case Diagram

In the Unified Modelling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, you'll use a set of specialized symbols and connectors. An effective use case diagram can help your team discuss and represent:

- Scenarios in which your system or application interacts with people, organizations, or external systems.
- Goals that your system or application helps those entities (known as actors) achieve.
- The scope of your system.

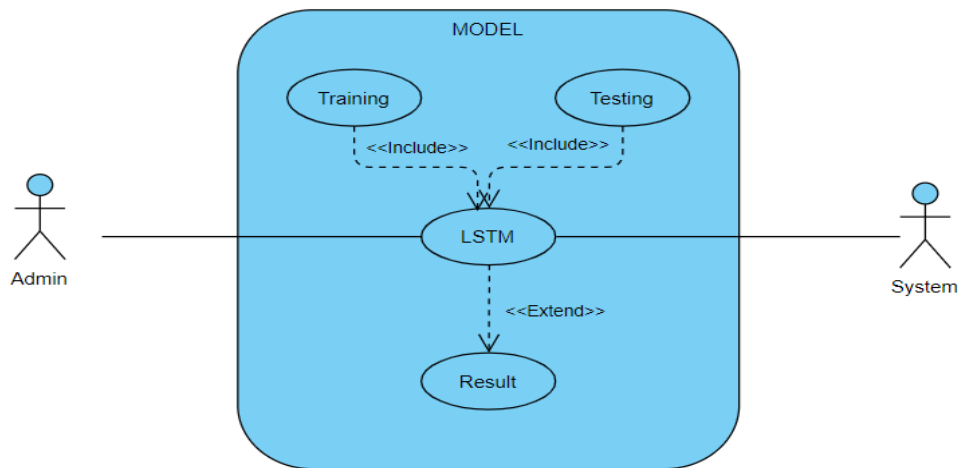


Fig. 5. Use Case Diagram for LSTM Model in System

6.2.2 CLASS DIAGRAM

A Class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

The class diagram is the main building block of object-oriented modelling. It is used for general conceptual modelling of the structure of the application, and for detailed modeling, translating the models into programming code. Class diagrams can also be used for data modelling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

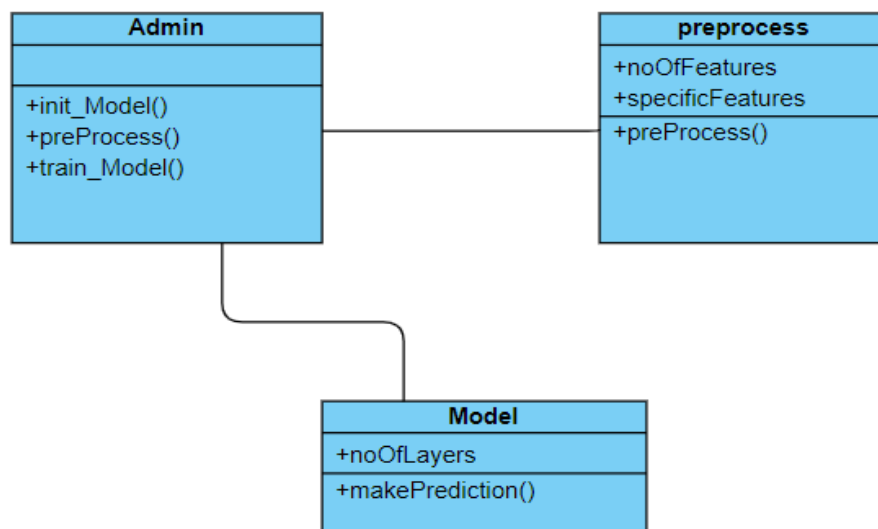


Fig. 6. Class diagram for LSTM Model

6.2.3 Sequence Diagram

A sequence diagram is a type of interaction diagram because it describes how and in what order a group of objects works together. These diagrams are used by software developers and business professionals to understand requirements for a new system or to document an existing process. Sequence diagrams are sometimes known as event diagrams or event scenarios.

Sequence diagrams can be useful references for businesses and other organizations. Try drawing a sequence diagram to:

- Represent the details of a UML use case.
- Model the logic of a sophisticated procedure, function, or operation.
- See how objects and components interact with each other to complete a process.
- Plan and understand the detailed functionality of an existing or future scenario.

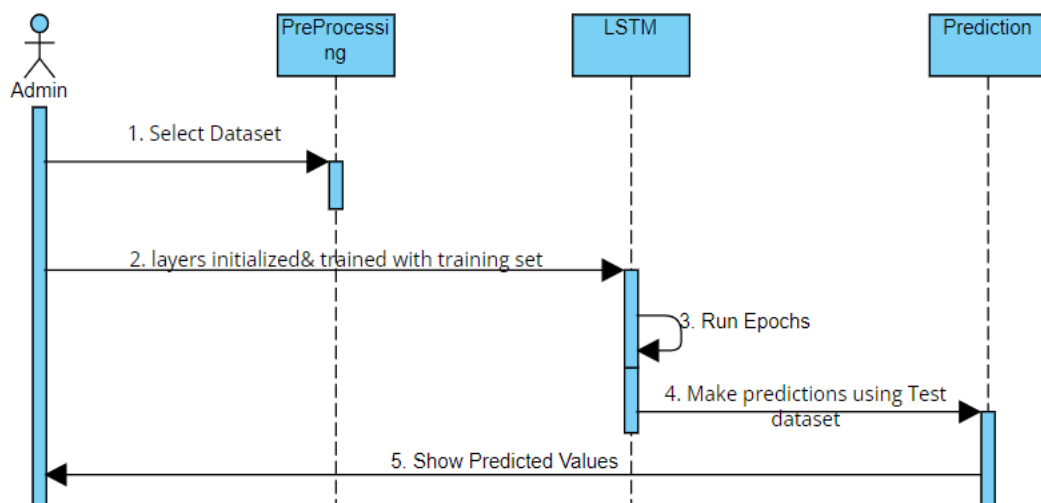


Fig. 7. Sequence Execution of Model

6.2.4 Activity Diagram

An activity diagram is a behavioral diagram i.e. it depicts the behavior of a system. An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed.

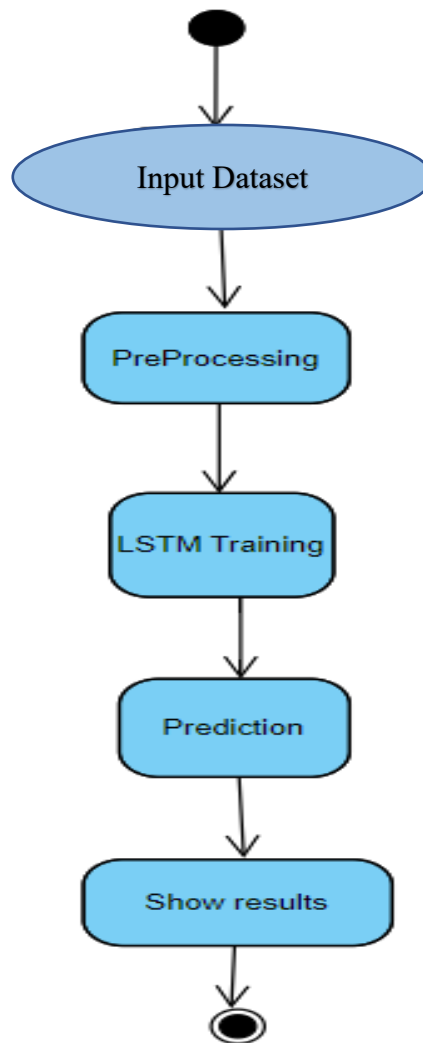


Fig. 8. Activity Diagram for LSTM Model

6.2.5 Collaboration Diagram

Collaboration diagrams are used to show how objects interact to perform the behavior of a particular use case, or a part of a use case. Along with sequence diagrams, collaborations are used by designers to define and clarify the roles of the objects that perform a particular flow of events of a use case. They are the primary source of information used to determining class responsibilities and interfaces.

The collaborations are used when it is essential to depict the relationship between the object. Both the sequence and collaboration diagrams represent the same information, but the way of portraying it quite different. The collaboration diagrams are best suited for analyzing use cases.

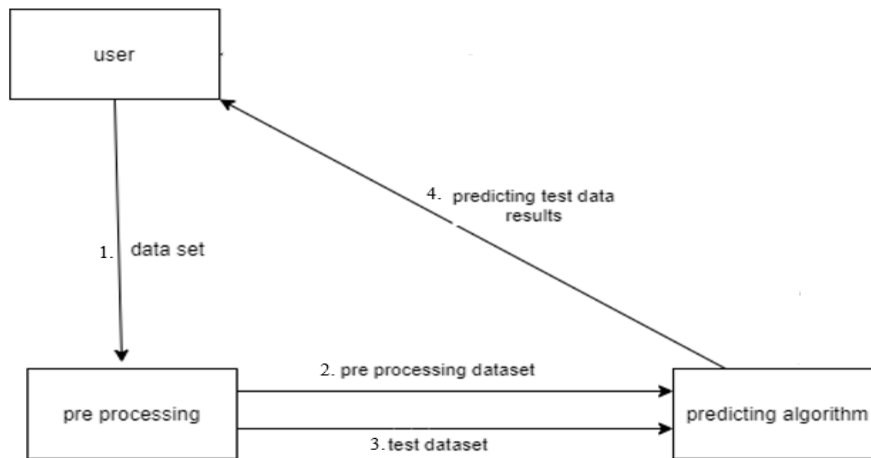


Fig 9. Data transfer between modules (collaboration)

CHAPTER 7: IMPLEMENTATION

7.1 SAMPLE CODE

```
#importing the necessary libraries & packages
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

#importing the dataset file with an extension of “.csv”
data = pd.read_csv('D:\Admin\Documents\BitCoin\BTC-USD.csv', date_parser = True)

# Data splitting
data_training = data[data['Date']< '2020-01-01'].copy()
data_training
data_test = data[data['Date']> '2020-01-01'].copy()
data_test

#Features Cleaning
training_data = data_training.drop(['Date', 'Adj Close'], axis = 1)
training_data.head()

#Normalization
scaler = MinMaxScaler()
training_data = scaler.fit_transform(training_data)

#RNN's LSTM initialization
model = Sequential()
model.add(LSTM(units = 50, activation = 'relu', return_sequences = True, input_shape =
(X_train.shape[1], 5)))
model.add(Dropout(0.2))

#Model fitting
model.compile(optimizer = 'adam', loss = 'mean_squared_error')
history= model.fit(X_train, Y_train, epochs = 20, batch_size =50, validation_split=0.1)

#Model Validation
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(len(loss))
```

CHAPTER 8: TESTING

8.1 TESTING METHODOLOGIES

There are different types of testing methods or techniques used as part of the software testing methodology. Some of the important testing methodologies are:

8.1.1 White Box Testing

White box testing (clear box testing, glass box testing, and transparent box testing or structural testing) uses an internal perspective of the system to design test cases based on internal structure. It requires programming skills to identify all paths through the software. The tester chooses test case inputs to exercise paths through the code and determines the appropriate outputs. While white box testing is applicable at the unit, integration and system levels of the software testing process, it is typically applied to the unit. While it normally tests paths within a unit, it can also test paths between units during integration, and between subsystems during a system level test.

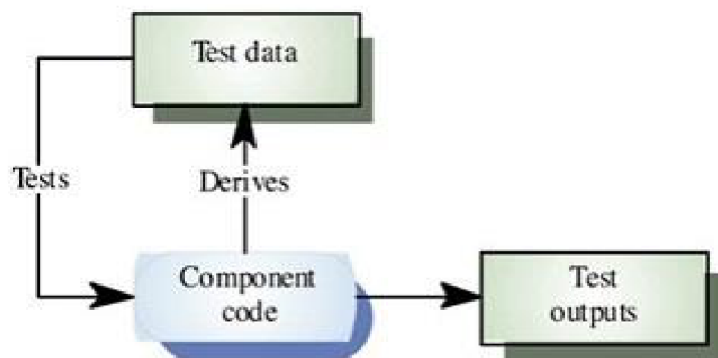


Fig. 10. White box Testing

Though this method of test design can uncover an overwhelming number of test cases, it might not detect unimplemented parts of the specification or missing requirements, but one can be sure that all paths through the test object are executed. Using white box testing we can derive test cases that:

- Guarantee that all independent paths within a module have been exercised at least once.
- Exercise all logical decisions on their true and false sides.
- Execute all loops at their boundaries and within their operational bounds.

- Execute internal data structure to assure their validity.

Advantages of White Box Testing

- To start the white box testing of the desired application there is no need to wait for user face (UI) to be completed. It covers all possible paths of code which will ensure a thorough testing.
- It helps in checking coding standards.
- Tester can ask about implementation of each section, so it might be possible to remove unused/deadlines of codes helps in reducing the number of test cases to be executed during the black box testing.
- As the tester is aware of internal coding structure, then it is helpful to derive which type of input data is needed to test the software application effectively.
- White box testing allows you to help in code optimization.

Disadvantages of White Boxing Testing

- To test the software application a highly skilled resource is required to carry out testing who has good knowledge of internal structure of the code which will increase the cost.
- Updating the test script is required if there is change in requirement too frequently.
- If the application to be tested is large in size, then exhaustive testing is impossible.
- It is not possible for testing each and every path/condition of software program, which might miss the defects in code.
- White box testing is a very expensive type of testing.
- To test each path or conditions may require different input conditions, so in order to test full application, the tester need to create range of inputs which may be a time consuming.

8.1.2 Black Box Testing

Black box testing focuses on the functional requirements of the software. It is also known as functional testing. It is a software testing technique whereby the internal workings of the item being tested are not known by the tester. For example, in a black box test on software design the tester only knows the inputs and what the expected outcomes should be and not how the program arrives at those outputs.

The tester does not ever examine the programming code and does not need any

further knowledge of the program other than its specifications. It enables us to derive sets of inputs that will fully exercise all functional requirements for a program.



Fig. 11. Black Box Testing

Black box testing is an alternative to white box technique. Rather it is a complementary approach that is likely to uncover a different class of errors in the following categories: -

- Incorrect or missing function.
- Interface errors.
- Performance errors.
- Initialization and termination errors.
- Errors in objects.

Advantages of Black Box Testing

- The test is unbiased as the designer and the tester are independent of each other.
- The tester does not need knowledge of any specific programming languages.
- The test is done from the point of view of the user, not the designer.
- Test cases can be designed as soon as the specifications are complete.

Disadvantages of Black Box Testing

- The test inputs need to be from large sample space. That is, from a huge set of data this will take time.
- Also, it is difficult to identify all possible inputs in limited testing time. So, writing test cases is slow and difficult.
- Chances are more that there will be unidentified paths during this testing.

8.2 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3 SYSTEM TESTING

This information contributes towards reducing the ambiguity about the system. For example, when deciding whether to release a product, the decision makers would need to know the state of the product including aspects such as the conformance of the product to requirements, the usability of the product, any known risks, the product's compliance to any applicable regulations.

Software testing enables making objective assessments regarding the degree of conformance of the system to stated requirements and specifications. System testing checks complete end-end scenarios, as a user would exercise the system. The system has to be tested for correctness of the functionality by setting it up in a controlled environment. System testing includes testing of functional and nonfunctional requirements. It helps to verify and validate the system. All components system should have been successfully unit tested and then checked for any errors after integration.

8.4 QUALITY ASSURANCE

Quality assurance consists of the auditing and reporting functions of management. The goal of quality assurance is to provide management with the data necessary to be informed about product quality, thereby gaining insight and confidence that the product quality is meeting its goals. This is an “umbrella activity” that is applied throughout the engineering process. Software quality assurance encompasses: -

- Analysis, design, coding and testing methods and tools.
- Formal technical reviews that are applied during each software engineering.

- Multi-tiered testing strategy.
- Control of software documentation and the change made to it.
- A procedure to ensure compliance with software development standards.
- Measurement and reporting mechanisms.

Quality Factors

An important objective of quality assurance is to track the software quality and assess the impact of methodological and procedural changes on improved software quality. The factors that affect the quality can be categorized into two broad groups:

- Factors that can be directly measured.
- Factors that can be indirectly measured

These factors focus on important aspects of a software product.

- Its operational characteristics
- Its ability to undergo changes
- Its adaptability to a new environment.
- Effectiveness or efficiency in performing its mission
- Duration of its use by its customer.

CHAPTER 9: RESULT & PERFORMANCE ANALYSIS

9.1 OUTPUT SNAPSHOTS

```
[49]: #Initialize the RNN
model = Sequential()
model.add(LSTM(units = 50, activation = 'relu', return_sequences = True, input_shape = (X_train.shape[1], 5)))
model.add(Dropout(0.2))

[50]: model.add(LSTM(units = 60, activation = 'relu', return_sequences = True))
model.add(Dropout(0.3))
model.add(LSTM(units = 120, activation = 'relu'))
model.add(Dropout(0.5))
model.add(Dense(units = 1))

[51]: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 60, 50)	11200
dropout_3 (Dropout)	(None, 60, 50)	0
lstm_4 (LSTM)	(None, 60, 60)	26640
dropout_4 (Dropout)	(None, 60, 60)	0
lstm_5 (LSTM)	(None, 120)	86880
dropout_5 (Dropout)	(None, 120)	0
dense_1 (Dense)	(None, 1)	121

=====
Total params: 124,841
Trainable params: 124,841
Non-trainable params: 0

Fig. 12. Model Initialization

```
[19]: loss = history.history['loss']
      val_loss = history.history['val_loss']

      epochs = range(len(loss))

      plt.figure()

      plt.plot(epochs, loss, 'b', label='Training loss')
      plt.plot(epochs, val_loss, 'r', label='Validation loss')
      plt.title("Training and Validation Loss")
      plt.legend()

      plt.show()
```

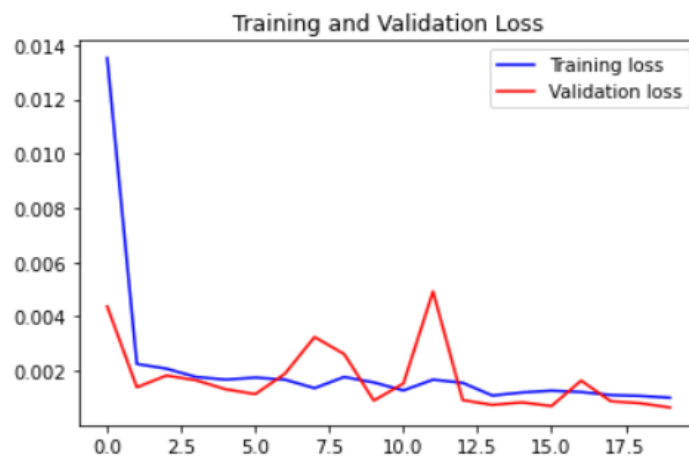


Fig 13. Model Validation

```
[29]: plt.figure(figsize=(14,5))
plt.plot(Y_test, color = 'red', label = 'Real Bitcoin Price')
plt.plot(Y_pred, color = 'green', label = 'Predicted Bitcoin Price')
plt.title('Bitcoin Price Prediction using RNN-LSTM')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.show()
```



Fig. 14. Model Prediction using 2 Layers

```
[30]: plt.figure(figsize=(14,5))
plt.plot(Y_test, color = 'red', label = 'Real Bitcoin Price')
plt.plot(Y_pred, color = 'green', label = 'Predicted Bitcoin Price')
plt.title('Bitcoin Price Prediction using RNN-LSTM')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.show()
```



Fig 15. Model Prediction using 3 Layer


```
[29]: plt.figure(figsize=(14,5))
plt.plot(Y_test, color = 'red', label = 'Real Bitcoin Price')
plt.plot(Y_pred, color = 'green', label = 'Predicted Bitcoin Price')
plt.title('Bitcoin Price Prediction using RNN-LSTM')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.show()
```



Fig.16. Model Prediction using 4 Layers

9.2 RESULT ANALYSIS

By Briefly examining the output generated by varying the number of layers in model, we noticed that model with 2 layers outperforms other layers model.

Also, model can predict the near values while model is trained with a dataset that having the null (or) missing values, but not better than prediction of trained model with having dataset that has no null (or) missing values.

CHAPTER 10: CONCLUSION & FUTURE WORK

Deep learning models such as the RNN and LSTM are evidently effective for Bitcoin prediction with the LSTM more capable for recognizing longer-term dependencies. However, a high variance task of this nature makes it difficult to transpire this into impressive validation results. As a result, it remains a difficult task. There is a fine line between overfitting a model and preventing it from learning sufficiently. Dropout is a valuable feature to assist in improving this. However, despite using Bayesian optimization to optimize the selection of dropout it still couldn't guarantee good validation results. Despite the metrics of sensitivity, specificity and precision indicating good performance, the actual performance of the SVM models based on error was significantly worse than the neural network models.

On the basis of future scope, we can extend this model for better use by developing the Web application or App.

REFERENCES

1. G. Solomon, "Project-Based learning: A Primer," Technology Learning, volume 23, Jan. 2003.
2. Yermack, D. (2017). Corporate governance and blockchains. Review of Finance 21(1), 7–31.
3. Huberman, G., J. D. Leshno, and C. C. Moallemi (2017). Monopoly without a monopolist: An economic analysis of the bitcoin payment system. Columbia Business School Research Paper No. 17-92.
4. Harvey, C. (2016). Cryptofinance. Working paper, Duke University. <http://ssrn.com/abstract=2438299>.
5. Harvey, C. (2014). Bitcoin myths and facts. Working paper, Duke University. Available at <http://ssrn.com/abstract=2479670>
6. Athey, S., I. Parashkevov, V. Sarukkai, and J. Xia (2016). Bitcoin pricing, adoption, and usage: Theory and evidence. Working paper, Stanford University.
7. K. J. Kim, "Financial time series forecasting using support vector machines," Neurocomputing, 55, 307 – 319, 2003.
8. W. Hunag, Y. Nakamori, S. Y. Wang, "Forecasting stock market movement direction with support vector machine," Computers and Operations Research 32, 2513-2522, 2005.
9. Neely, C. J., D. E. Rapach, J. Tu, and G. Zhou (2014). Forecasting the equity risk premium: the role of technical indicators. Management Science 60 (7), 1772–1791.
10. <http://cs229.stanford.edu/proj2014/Isaac%20Madan,%20Shaurya%20Saluja,%20Aojia%20Zhao,Automated%20Bitcoin%20Trading%20via%20Machine%20Learning%20Algorithms.pdf>.
11. Data collection reference, "[BTC-USD 39,850.71 -2650.04 -6.24 : Bitcoin USD - Yahoo Finance](#)"